

DISTRIBUIRANI SISTEMI - I KOLOKVIJUM

Teorija:

1. Objasniti transparentnost lokacije i transparentnost otkaza.
2.
 - a. Navesti i objasniti načine prenosa parametara kod RPC.
 - b. Šta se dešava nakon unmarshalling-a na serveru?
 - A) Šalje se rezultat odmah
 - B) Poziva se serverska funkcija
 - C) Briše se zahtev
 - D) Restartuje se server
3.
 - a. Na koji način klijent u Sun RPC dobija informacije potrebne za povezivanje sa udaljenim servisom? Objasniti postupak povezivanja. Zašto server koristi static promenljive za rezultat?
 - b. Napisati SunRPC interfejs koji treba da omogući registrovanim biračima da glasaju za određene kandidate i pregledaju rezultate glasanja. Server treba da implementira metodu **vote** kojom klijent prosleđuje identifikator kandidata i jedinstveni broj birača, pri čemu server proverava da li je birač već glasao i evidentira glas ukoliko je glasanje validno, vraćajući informaciju o uspešnosti operacije. Takođe, potrebno je implementirati metodu **getResult** za zadati identifikator kandidata vraća broj glasova koje je taj kandidat osvojio i koje mesto po broju glasova zauzima.
4.
 - a. Šta predstavlja UUID kod DCE RPC i kako se generiše? Šta se dešava nakon poziva idl primer.idl?
 - b. U DCE RPC sistemu, poređati sledeće korake redosledom kojim se izvršavaju prilikom poziva udaljene procedure:
 - a) Izvršenje udaljene procedure
 - b) Klijent preko binder/directory servisa traži UUID interfejs
 - c) Marshalling parametara na strani klijenta
 - d) Klijent šalje RPC zahtev serveru
5. Šta je referenca udaljenog objekta i koje informacije ona sadrži? Kako se ona koristi pri povezivanju klijenta i servera i koja je uloga klase UnicastRemoteObject u tome?
6. Koje su uloge sesije u JMS sistemu?
 - a. Kreiranje proizvođača i potrošača poruka
 - b. Kreiranje poruka
 - c. Upravljanje transakcijama poruka
 - d. Uspostavljanje fizičke mrežne konekcije

Zadaci:

1. U .NET-u, koristeći gRPC, napisati servis za upravljanje bibliotekom knjiga. Servis treba da omogući klijentima dodavanje, pretragu i iznajmljivanje knjiga. Svaka knjiga ima polja: *Id*, *title*, *author* i *available*. Servis treba da podržava sledeće operacije:
 - a. **AddBook**: Dodaje novu knjigu u sistem.
 - b. **SearchByAuthor**: Prihvata ime autora i vraća tok podataka koji sadrži sve knjige tog autora sa svim informacijama.
 - c. **BorrowBook**: Prihvata *Id* knjige i postavlja polje *available* na *false*. Ukoliko knjiga nije dostupna, procedura vraća odgovarajuću grešku.

Definisati i proto fajl (*library.proto*) koji daje specifikaciju servisa i poruka.

2. Korišćenjem Java RMI tehnologije napisati simulaciju sistema za praćenje cena akcija. Implementirati klasu *Stock* sa atributima (*int Id*, *String companyName*, *double price*). Klasa treba da ima metodu koja vraća *String* literal sa *Id*-jem, nazivom kompanije i trenutnom cenom. Implementirati klasu *StockService* koja klijentima omogućava:
 - a. Metodu koja vraća listu svih akcija u sistemu.
 - b. Metodu za promenu cene akcije po *Id*-ju.
 - c. Pretplatu klijenata na promene cene akcija. Callback interfejs sadrži potpis udaljene metode *priceChanged(int Id, double newPrice)*.

U konstruktoru klase *StockService* kreirati dva objekta klase *Stock* i smestiti ih u odgovarajuću strukturu podataka. Implementirati serversku klasu *Server* koja registruje *StockService* u RMI registar na portu 5099. Klijentska klasa *Client* treba da prikaže sve akcije, pretplati se na promene, implementira callback interfejs i ispiše obaveštenje pri promeni cene. Klijentska klasa *AdminClient* treba da promeni cenu jedne od akcija.